

Password Security Demo

A beginner-friendly cybersecurity project demonstrating how passwords are protected using hashing and salting, why weak passwords are dangerous, and how attackers attempt to compromise accounts through password attacks.

Overview

Passwords are one of the most common authentication mechanisms used today. Unfortunately, weak passwords and poor storage practices are still responsible for many security breaches.

This project was created to demonstrate:

- Weak password identification
- Password hashing with SHA-256
- Salting techniques
- Password strength evaluation
- Common password attacks
- Security best practices

The purpose of this project is educational and focuses on defensive cybersecurity concepts.

Objectives

- Understand why weak passwords are risky.
- Learn how hashing protects passwords.
- Demonstrate how salting prevents identical hashes.
- Evaluate password strength using simple criteria.
- Explain common password attacks and mitigation methods.

Tools Used

- Python 3
- hashlib
- os
- Re

Project Structure

```
password-security-demo/  
|  
├── weak_passwords.py  
├── hashing_demo.py  
├── salting_demo.py  
├── password_strength.py  
└── screenshots/
```

Part 1: Weak Password Demonstration

Many users still rely on predictable passwords such as:

- 123456
- password
- qwerty
- admin
- welcome123

These passwords are commonly included in dictionary attacks and are easily guessed.

Example

Input:

password

Output:

✗Weak password detected

Explanation

Attackers frequently use lists containing millions of leaked passwords. If a user chooses one of these passwords, the account can often be compromised within seconds.

Part 2: Password Hashing

Instead of storing plaintext passwords, systems should store hashes. Hashing converts a password into a fixed-length value that cannot easily be reversed.

Example:

Password:
Cyber123!

SHA-256 Hash:
24db6d6f0eec...

How Hashing Works

Cyber123!

↓

SHA-256

↓

24db6d6f0eec...

When a user logs in:

1. The password is entered.
2. The system hashes the input.
3. The resulting hash is compared with the stored hash.
4. If both hashes match, authentication succeeds.

Part 3: Salting

A salt is a random value added to the password before hashing.

Without salting:

Password:
Cyber123!

Hash:
24db6d6f0eec...

Another user with the same password would produce the exact same hash.

With salting:

First Run

Salt:
af8b71...
Hash:
2e1c7f...

Second Run

Salt:
a927de...
Hash:
90d5f8...

Even though the password remains identical, the resulting hashes are different.

Why Salting Matters

Salting helps defend against:

- Rainbow table attacks
- Hash comparison attacks
- Large-scale password database leaks

Part 4: Password Strength Checker

The password strength checker evaluates:

- Minimum length
- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Example

Weak password:
password123

Output:
Weak

Strong password:
MySecurePassword2026!

Output:
Very Strong

Common Password Attacks

Brute Force Attack

An attacker tries every possible combination until the correct password is found.

Defense

- Long passwords
- Account lockout policies
- Multi-factor authentication (MFA)

Dictionary Attack

Attackers use databases containing commonly used passwords.

Defense

- Avoid common passwords
- Use passphrases
- Increase password complexity

Rainbow Table Attack

Attackers use precomputed hash databases to recover passwords.

Defense

- Salting
- bcrypt
- Argon2

Credential Stuffing

Attackers reuse passwords leaked from other websites.

Defense

- Unique passwords
- Password managers
- MFA

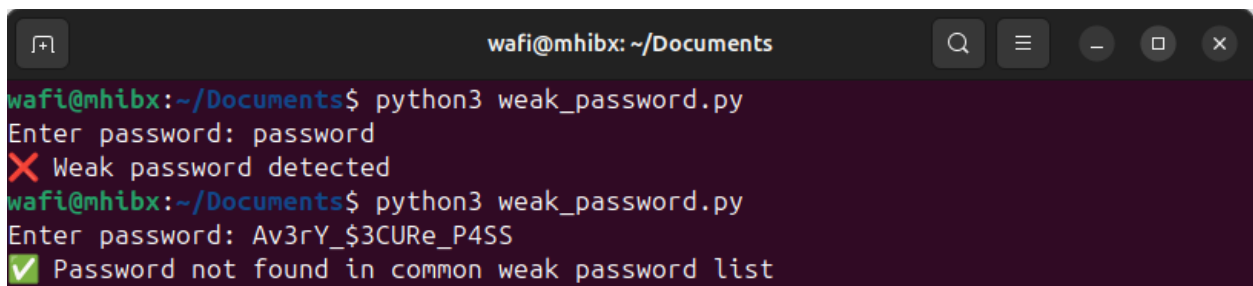
Security Best Practices

- ✓ Use passwords with at least 12 characters.
- ✓ Include uppercase, lowercase, numbers, and symbols.
- ✓ Use a unique password for every account.
- ✓ Enable multi-factor authentication.
- ✓ Use password managers.
- ✓ Store passwords using modern algorithms such as:

- bcrypt
- scrypt
- Argon2

Screenshots

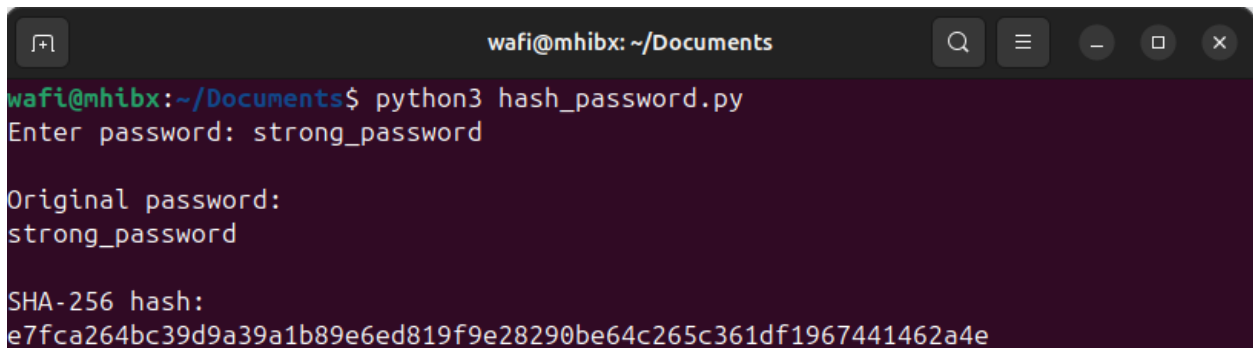
Weak Password Detection



```
wafi@mhibx: ~/Documents
wafi@mhibx:~/Documents$ python3 weak_password.py
Enter password: password
❌ Weak password detected
wafi@mhibx:~/Documents$ python3 weak_password.py
Enter password: Av3rY_$3CURE_P4SS
✅ Password not found in common weak password list
```

A terminal window titled 'wafi@mhibx: ~/Documents' showing two runs of a Python script 'weak_password.py'. The first run takes 'password' as input and outputs '❌ Weak password detected'. The second run takes 'Av3rY_\$3CURE_P4SS' as input and outputs '✅ Password not found in common weak password list'.

SHA-256 Hashing Demo



```
wafi@mhibx: ~/Documents
wafi@mhibx:~/Documents$ python3 hash_password.py
Enter password: strong_password

Original password:
strong_password

SHA-256 hash:
e7fca264bc39d9a39a1b89e6ed819f9e28290be64c265c361df1967441462a4e
```

A terminal window titled 'wafi@mhibx: ~/Documents' showing a run of a Python script 'hash_password.py'. It prompts for a password, shows the original password 'strong_password', and then displays the SHA-256 hash: 'e7fca264bc39d9a39a1b89e6ed819f9e28290be64c265c361df1967441462a4e'.

Salting Demo

```
wafi@mhibx: ~/Documents
wafi@mhibx:~/Documents$ python3 salt_hash_password.py
Enter password: strong_password
Salt: b22257d6b02da6d8a9586d4fcd135605
Hash: aa5ea77be29616dfda6555d919434fc6bc43cfd0defc24a1803878af2acad94
wafi@mhibx:~/Documents$ python3 salt_hash_password.py
Enter password: strong_password
Salt: 2ec1ea9dd1f2df95107caca1085fb80c
Hash: 770bfec18adeaa6021928749c4c7abb76612743549a1d493d360cd2eb7de5e56
```

Password Strength Checker

```
wafi@mhibx: ~/Documents
wafi@mhibx:~/Documents$ python3 strong_check_password.py
Enter password: password
Password Strength: Weak
wafi@mhibx:~/Documents$ python3 strong_check_password.py
Enter password: strong_password
Password Strength: Good
wafi@mhibx:~/Documents$ python3 strong_check_password.py
Enter password: $tR0n6_P@5Sw0Rd
Password Strength: Very Strong
```

Key Concepts Demonstrated

- Authentication security
- Password management
- Cryptographic hashing
- Salting techniques
- Defensive cybersecurity principles
- Python scripting

Skills Developed

- Python Programming
- Cybersecurity Fundamentals
- Password Security
- Cryptography Basics
- Technical Documentation
- Security Awareness

Conclusion

This project demonstrates the importance of secure password storage and highlights how weak passwords can expose users to attacks. By implementing hashing and salting techniques and understanding common attack methods, organizations can significantly improve authentication security.

Future Improvements

- Implement bcrypt instead of SHA-256.
- Compare SHA-256 with bcrypt and Argon2.
- Build a GUI version using Tkinter.
- Add password entropy calculations.
- Integrate with a simple login system.
- Create a web-based password checker using Flask.